

Question No:3

DS

Session

Sorting

Question Information

Level 1

Challenge 13

Problem Description:

Siva has several containers, each with a number of fruits in it. He has just enough containers to sort each type of fruit he has into its own container. Siva wants to sort the fruits using his sort method.

Siva wants to perform some number of swap operations such that:

Each container contains only fruits of the same type.

No two fruits of the same type are located in different containers.

Function Description

organizingContainers has the following parameter(s):

int container[n][m]: a two dimensional array of integers that represent the number of balls of each color in each container

Constraints:

$1 \leq q \leq 10$

$1 \leq n \leq 100$

$0 \leq \text{containers}[i][j] \leq 10^9$

Input Format:

The first line contains an integer 'q', the number of queries.

Each of the next 'q' sets of lines is as follows:

The first line contains an integer 'n', the number of containers (rows) and ball types (columns).

Each of the next 'n' lines contains 'n' space-separated integers describing row containers[i].

Output Format:

For each query, print Possible on a new line if David can satisfy the conditions above for the given matrix. Otherwise, print Impossible.

▼ Logical Test Cases

Test Case 1

INPUT (STDIN)

2
2
1 1
1 1
2
0 2
1 1

EXPECTED OUTPUT

Possible
Impossible

Test Case 2

INPUT (STDIN)

2
2
2 1
1 2
2
1 2
2 1

EXPECTED OUTPUT

Possible
Possible

▼ Mandatory Test Cases

Test Case 1

KEYWORD

void insertionSort(long int
*p,long int n)

Test Case 2

KEYWORD

for(i=0;i<n;i++)

Test Case 3

KEYWORD

while(q--)

▼ Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

5

Test Case 2

TOKEN COUNT

450

Test Case 3

NLOC

63

Code:

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  int main() {
4      int q;
5      scanf("%d", &q);
6      while (q--) {
7          int n;
8          scanf("%d", &n);
9
10         int container[n][n];
11         long long rowSum[n];
12         long long colSum[n];
13
14         // Initialize sums
15         for (int i = 0; i < n; i++) {
16             rowSum[i] = 0;
17             colSum[i] = 0;
18         }
19
20         // Read matrix and calculate row and column sums
21         for (int i = 0; i < n; i++) {
22             for (int j = 0; j < n; j++) {
23                 scanf("%d", &container[i][j]);
24
25                 rowSum[i] += container[i][j];
26                 colSum[j] += container[i][j];
27             }
28         }
```

```

30 // Sort row sums
31 for (int i = 0; i < n - 1; i++) {
32     for (int j = i + 1; j < n; j++) {
33         if (rowSum[i] > rowSum[j]) {
34             long long temp = rowSum[i];
35             rowSum[i] = rowSum[j];
36             rowSum[j] = temp;
37         }
38     }
39 }
40 // Sort column sums
41 for (int i = 0; i < n - 1; i++) {
42     for (int j = i + 1; j < n; j++) {
43         if (colSum[i] > colSum[j]) {
44             long long temp = colSum[i];
45             colSum[i] = colSum[j];
46             colSum[j] = temp;
47         }
48     }
49 }
50 // Compare row sums and column sums
51 int possible = 1;
52
53 for (int i = 0; i < n; i++) {
54     if (rowSum[i] != colSum[i]) {
55         possible = 0;
56         break;
57     }
58 }
59 if (possible)
60     printf("Possible\n");
61 else
62     printf("Impossible\n");
63 }
64 return 0;
65 }

```

Output:

Test Case-1:

```
2
2
1 1
1 1
Possible
2
0 2
1 1
Impossible
```

Test Case-2:

```
2
2
2 1
1 2
Possible
2
1 2
2 1
Possible
```

Keshav Kumar Rauniyar
CH.SC.U4CSE25236

